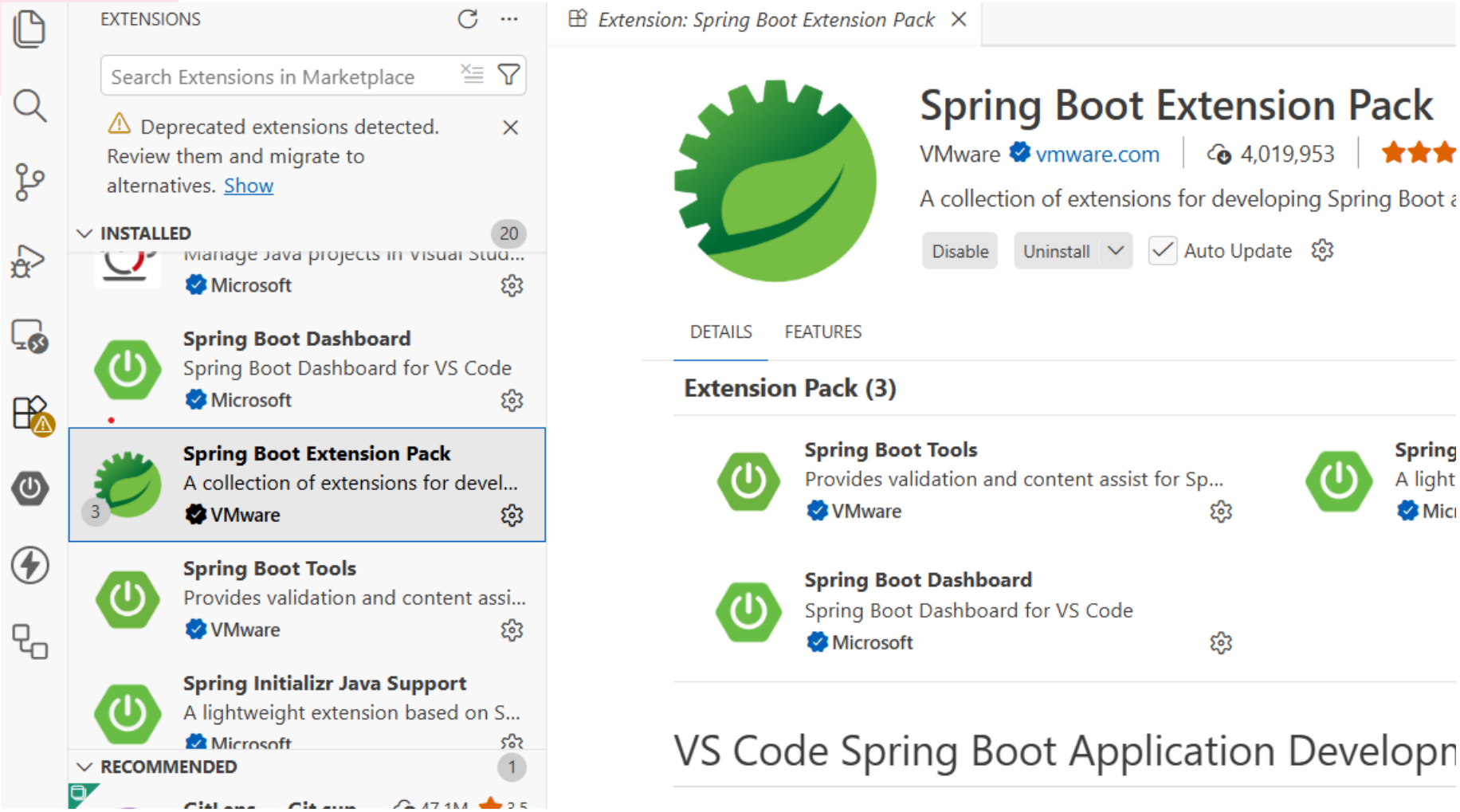
Spring Boot

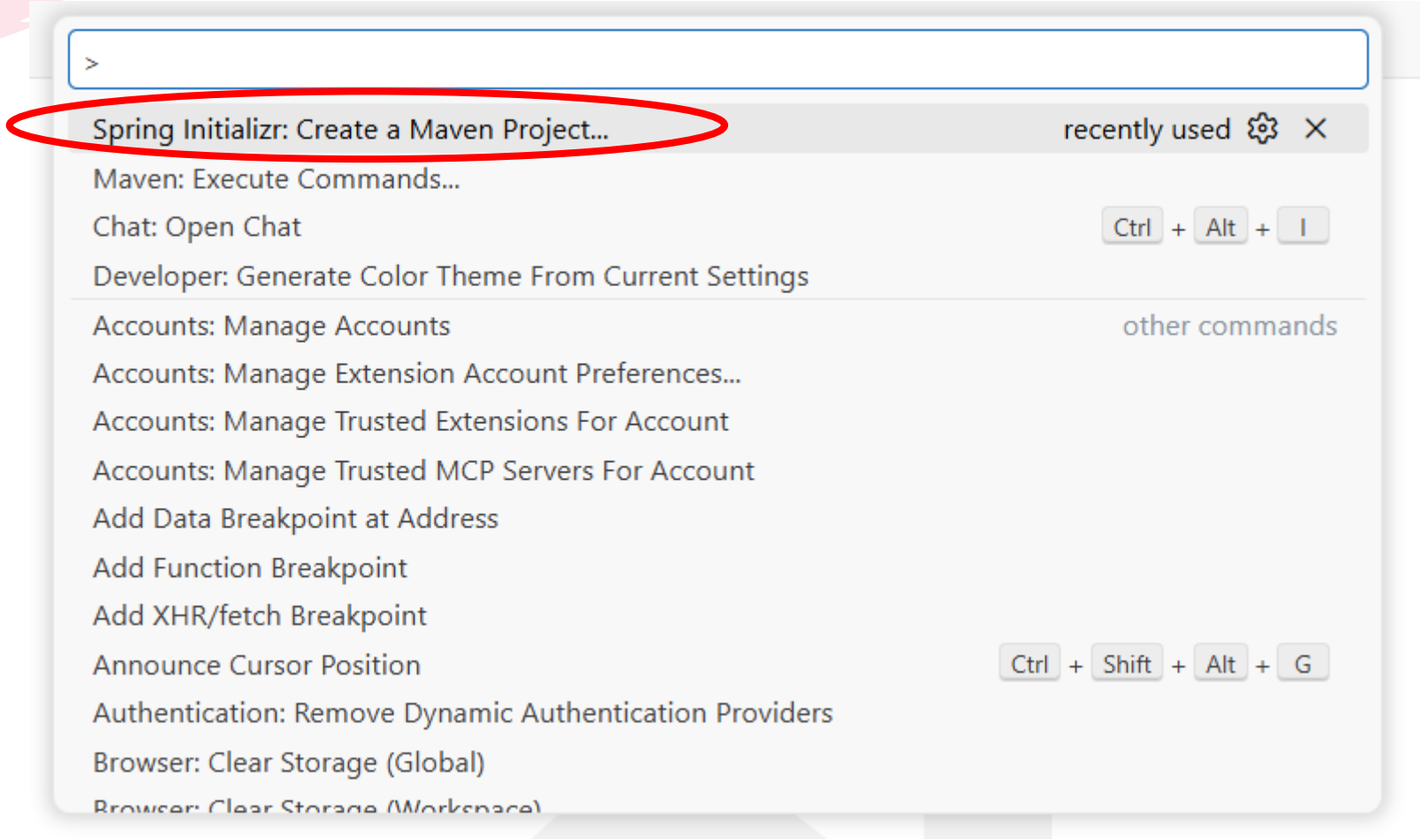
Persistência Api-Rest

PAULO R T CÂNDIDO

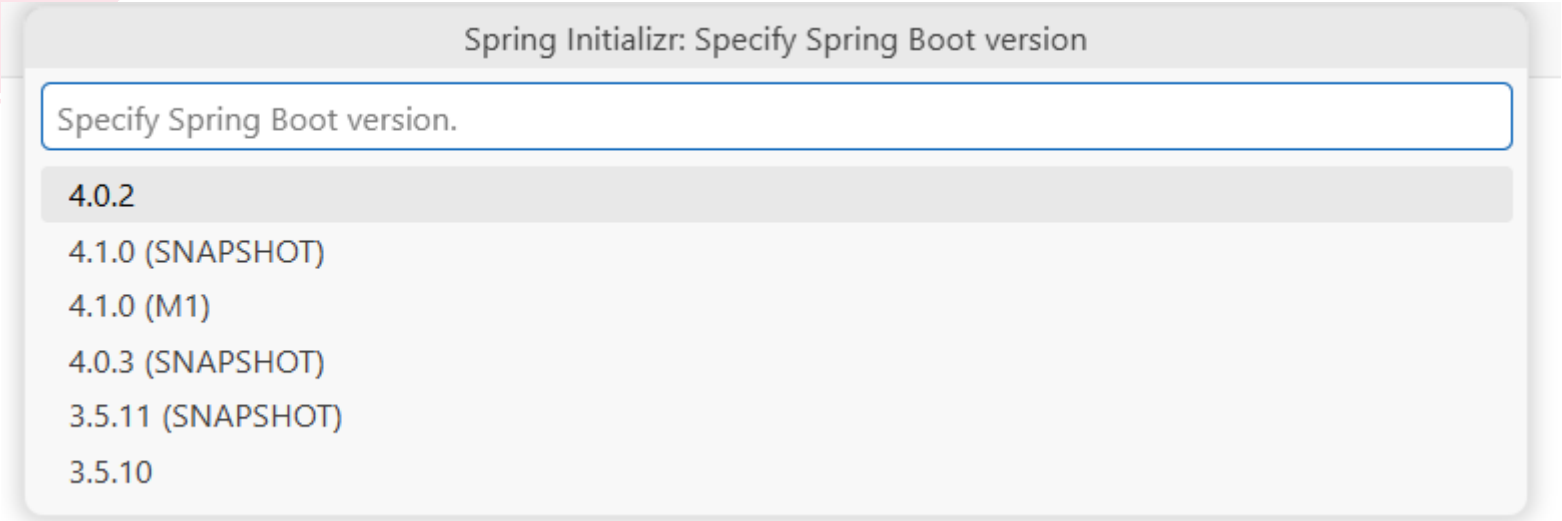
VSC - Spring Boot Extension Pack



Criar Projeto



Versões

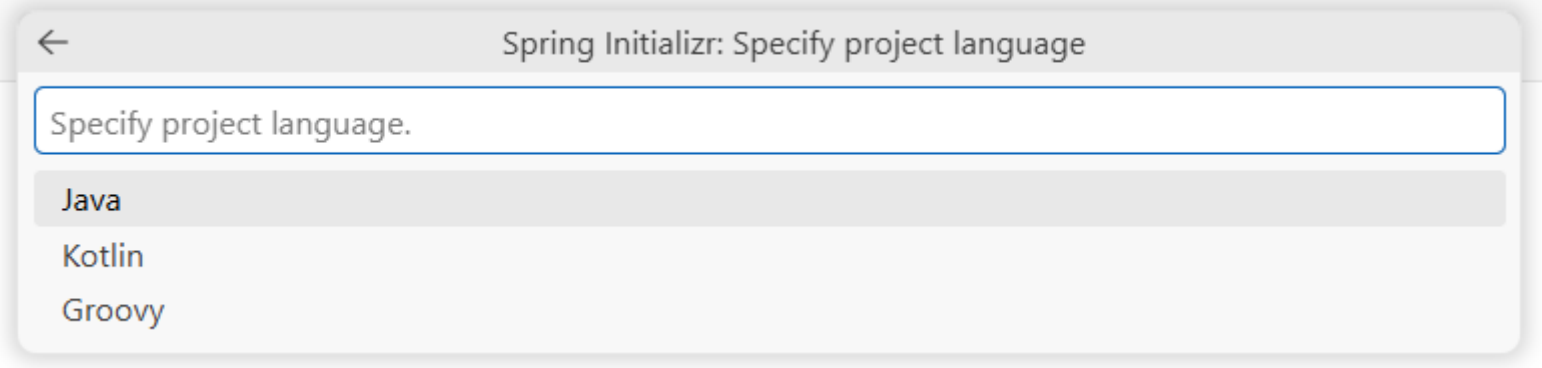


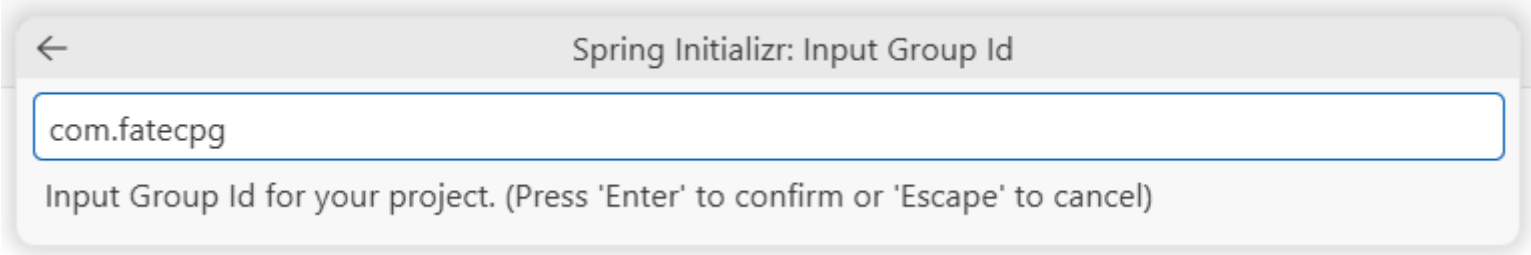
Escolha a versão "GA" (General Availability)

Sempre escolha as versões que **não possuem sufixo** (Snapshot, Milestone ou RC).

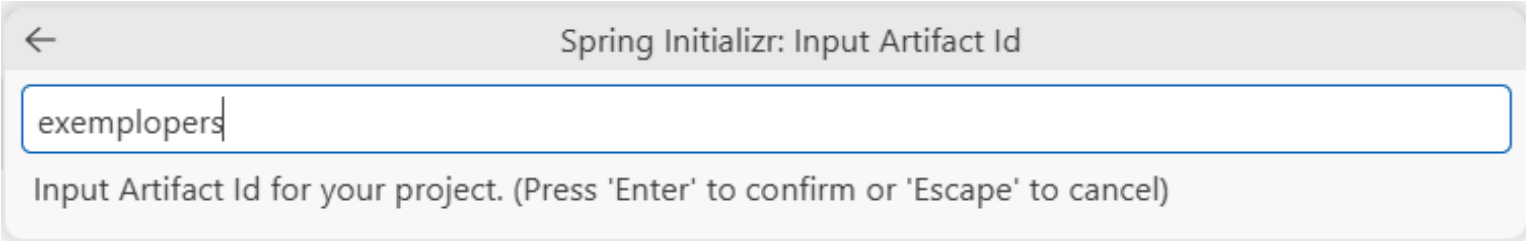
- **Snapshot:** Versão em desenvolvimento, instável. Fuja dela para projetos sérios.
- **M (Milestone) / RC (Release Candidate):** Versões de teste antes do lançamento oficial.
- **GA (ou sem sufixo):** É a versão estável, recomendada para uso real.

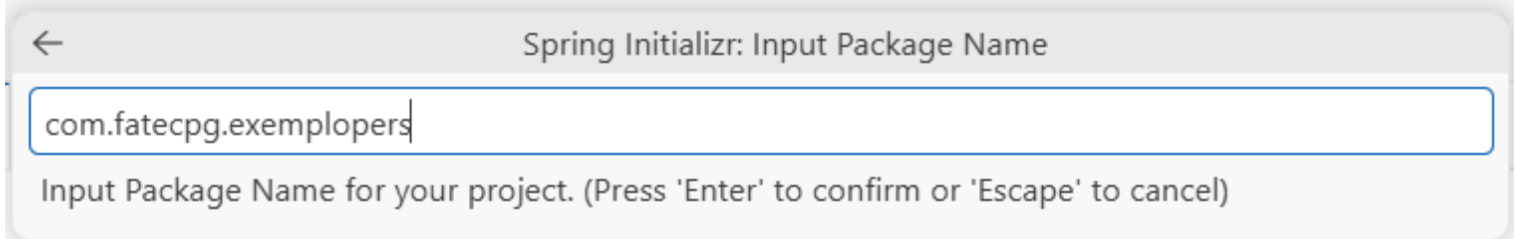
Linguagem e Group Id



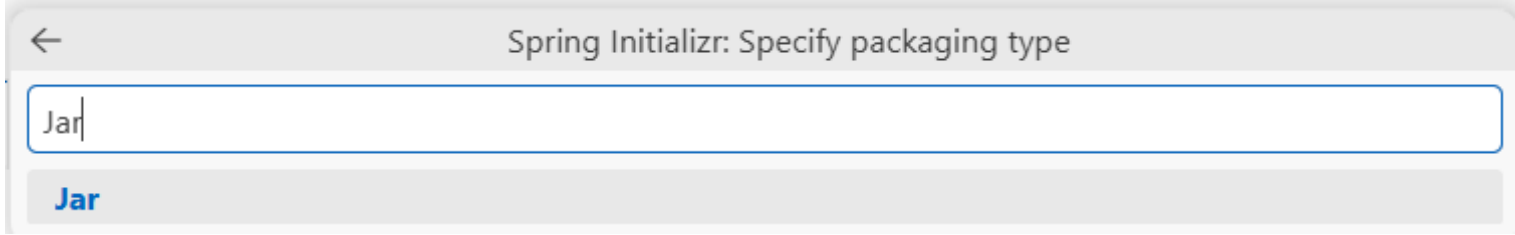


Nome do projeto e pacote (eq namespace)



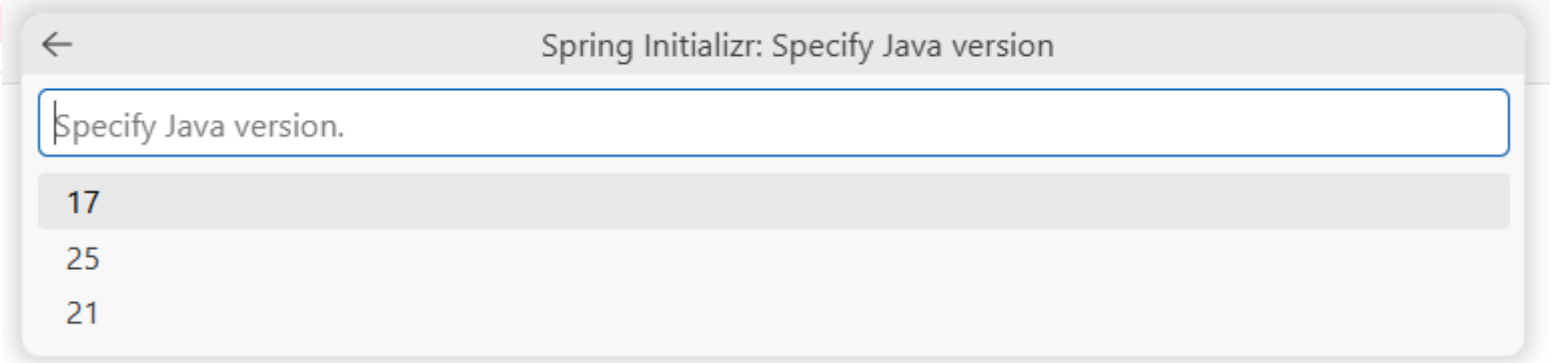


Compactação para distribuição

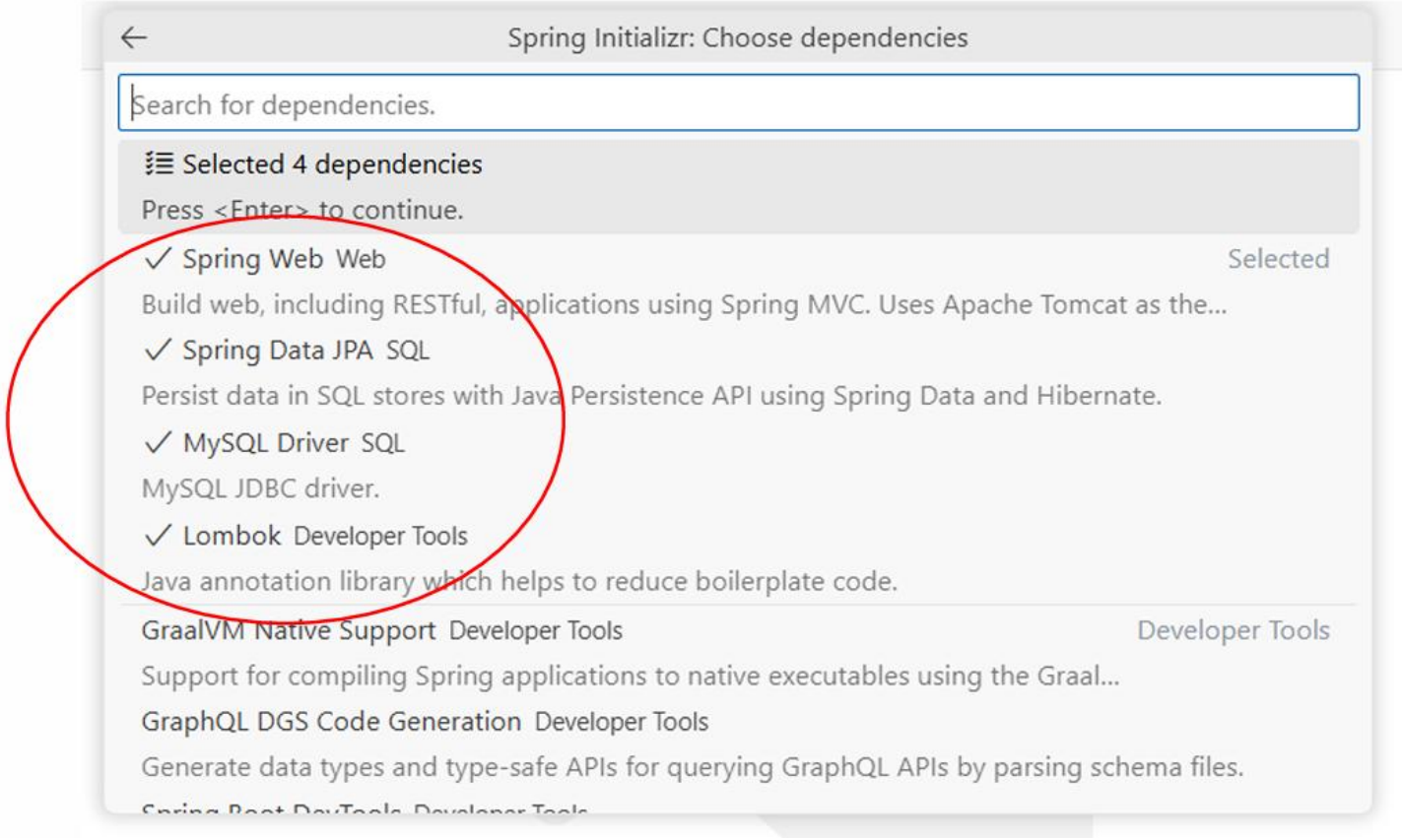


Característica	JAR (Spring Boot)	WAR
Servidor Web	Embutido (Incluso no pacote)	Externo (Instalado no SO)
Execução	Direta via terminal	Gerenciada pelo servidor
Nuvem/Docker	Recomendado (Cloud Native)	Mais complexo de configurar
Estrutura	Focada em classes e libs	Possui pasta `WEB-INF` obrigatória

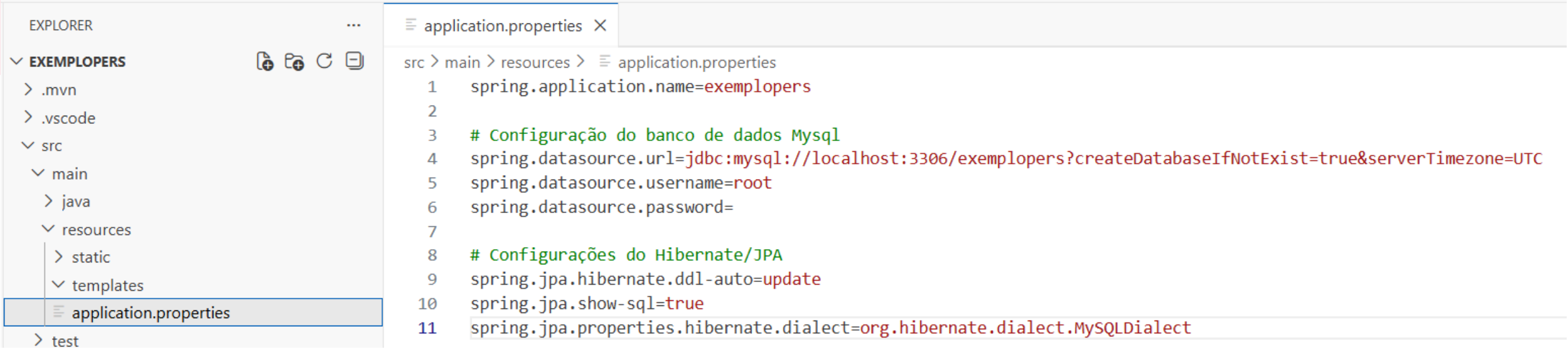
Versão Java e dependências



Obs.: usar 17

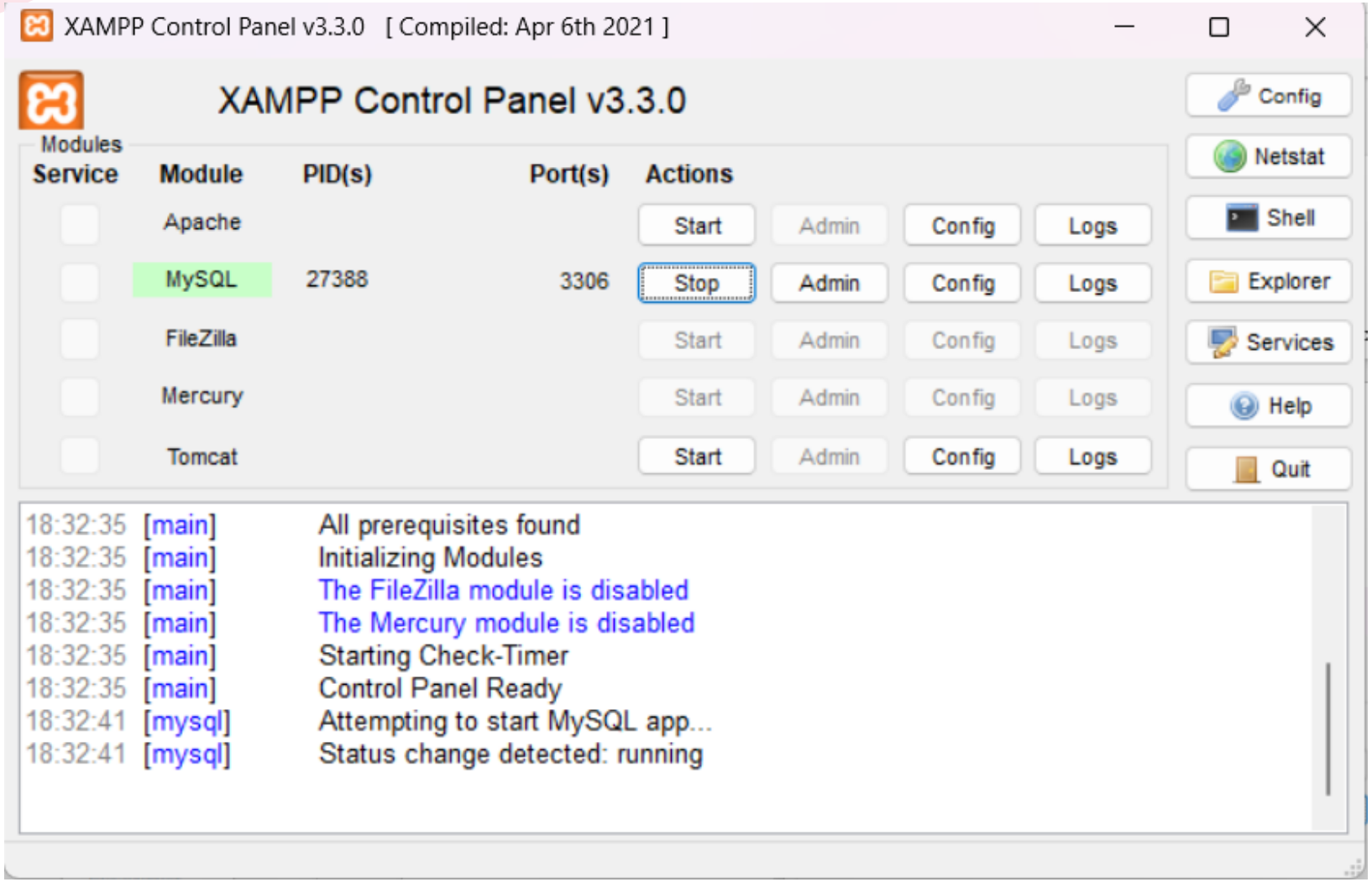


Configuração MySQL e JPA

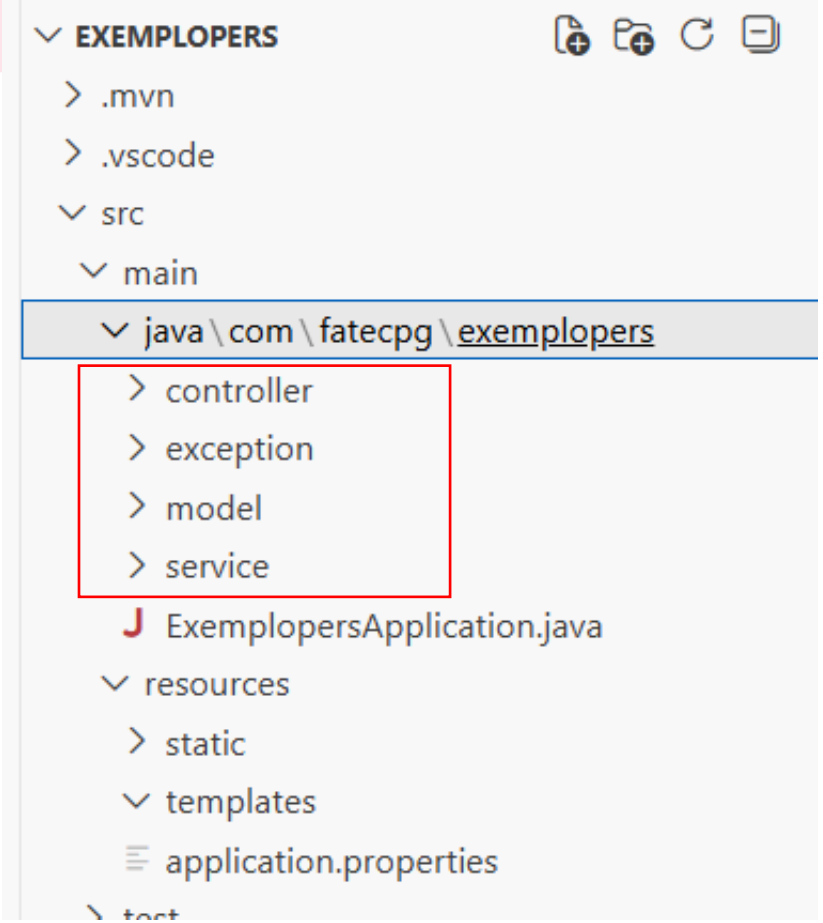



```
spring.application.name=exemplopers


# Configuração do banco de dados Mysql
spring.datasource.url=jdbc:mysql://localhost:3306/exemplopers?createDatabaseIfNotExist=true&serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=


# Configurações do Hibernate/JPA
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
```

Ligar MySQL


XAMPP Control Panel v3.3.0 [Compiled: Apr 6th 2021]
XAMPP Control Panel v3.3.0
Modules
Service
Module
PID(s)
Port(s)
Actions
Apache
Start
Admin
Config
Logs
MySQL
27388
3306
Stop
Admin
Config
Logs
FileZilla
Start
Admin
Config
Logs
Mercury
Start
Admin
Config
Logs
Tomcat
Start
Admin
Config
Logs
Config
Netstat
Shell
Explorer
Services
Help
Quit
18:32:35 [main] All prerequisites found
18:32:35 [main] Initializing Modules
18:32:35 [main] The FileZilla module is disabled
18:32:35 [main] The Mercury module is disabled
18:32:35 [main] Starting Check-Timer
18:32:35 [main] Control Panel Ready
18:32:41 [mysql] Attempting to start MySQL app...
18:32:41 [mysql] Status change detected: running

Criar estrutura de pastas



- Controller: implementam a interface com o cliente (pessoa ou software). Recebem as requisições e retornam as respostas. Não realizam cálculo ou acesso ao banco de dados, que ficam a cargo dos serviços.
- Serviços: realizam cálculo ou acesso ao banco de dados.
- Model (ou Entity): onde são definidos os objetos persistentes. Objetos com os dados do negócio e que, portanto, precisam ser armazenados no banco de dados (MySQL neste exemplo).
- Exception: tratamento de exceções (falhas técnicas ou de negócio)

Classe Pessoa POJO (Plain Old Java Object)

EXPLORER

- EXEMPLOPERS
  - .mvn
  - .vscode
  - src
    - main
      - java \ com \ fatecpg \ exemplopers
        - controller
        - exception
        - model
          - Pessoa.java
        - service
        - ExemplopersApplication.java
      - resources
        - static
        - templates
        - application.properties
    - test
  - target
  - .gitattributes
  - .gitignore
  - HELP.md
  - mvnw
  - mvnw.cmd
  - pom.xml

application.properties | Pessoa.java

src > main > java > com > fatecpg > exemplopers > model > Pessoa.java > ...

```java
package com.fatecpg.exemplopers.model;

public class Pessoa {
    private long id;
    private String nome;

    public Pessoa() {
    }

    public Pessoa(long id, String nome) {
        this.id = id;
        this.nome = nome;
    }

    public long getId() {
        return id;
    }

    public void setId(long id) {
        this.id = id;
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }
}
```

Lombok – gerando get/set, construtores e mais

```java
package com.fatecpg.exemplopers.model;

import lombok.*;

@Getter  // gera métodos get
@Setter  // gera métodos set
@NoArgsConstructor  // gera construtor sem parâmetro
@AllArgsConstructor // gera construtor com parâmetros
@ToString(onlyExplicitlyIncluded = true) // para definição de toString()
@EqualsAndHashCode(onlyExplicitlyIncluded = true) // para definição de equal e hashCode
public class Pessoa {

    @ToString.Include  // inclui id no resultado de toString()
    @EqualsAndHashCode.Include // equals e hashCode baseados apenas no id
    private long id;

    @ToString.Include // inclui nome no resultado de toString()
    private String nome;

}
```

Transformando em Entity

```java
package com.fatecpg.exemplopers.model;

import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import lombok.*;

@Entity  // Indica que esta classe é uma entidade JPA, ou seja, será mapeada para uma tabela no banco de dados.
@Table(name = "pessoas") // Especifica o nome da tabela no banco de dados que esta entidade irá mapear.
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@ToString(onlyExplicitlyIncluded = true)
@EqualsAndHashCode(onlyExplicitlyIncluded = true)
public class Pessoa {

    @Id // Indica que o campo "id" é a chave primária da entidade.
    @GeneratedValue(strategy = GenerationType.IDENTITY) // Especifica que o valor do campo "id" será gerado automaticamen
    @Column(nullable = false) // Especifica que a coluna "id" é obrigatória (não pode ser nula)
    @ToString.Include
    @EqualsAndHashCode.Include
    private long id;

    @Column(nullable = false, unique = true) // Especifica que a coluna "nome" é obrigatória e única (sem repetições)
    @ToString.Include
    private String nome;
}
```

Criar Repository – Interface com BD

Todas as operações com BD são feitas por um Repository. No caso operações de leitura e gravação de objetos do tipo Pessoa (classe). As operações básicas já estão prontas, não precisam ser programadas.

```
package com.fatecpg.exemplopers.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import com.fatecpg.exemplopers.model.Pessoa;

@Repository
public interface PessoaRepository extends JpaRepository<Pessoa, Long> {
}
```

Classe de objetos

Tipo da coluna ID

Service

```java
package com.fatecpg.exemplopers.service;

import org.springframework.stereotype.Service;
import com.fatecpg.exemplopers.model.Pessoa;
import com.fatecpg.exemplopers.repository.PessoaRepository;

@Service
public class PessoaService {

    private final PessoaRepository pessoaRepository;

    public PessoaService(PessoaRepository pessoaRepository) {
        this.pessoaRepository = pessoaRepository;
    }

    public Pessoa salvarPessoa(Pessoa pessoa) {
        return pessoaRepository.save(pessoa);
    }

}
```

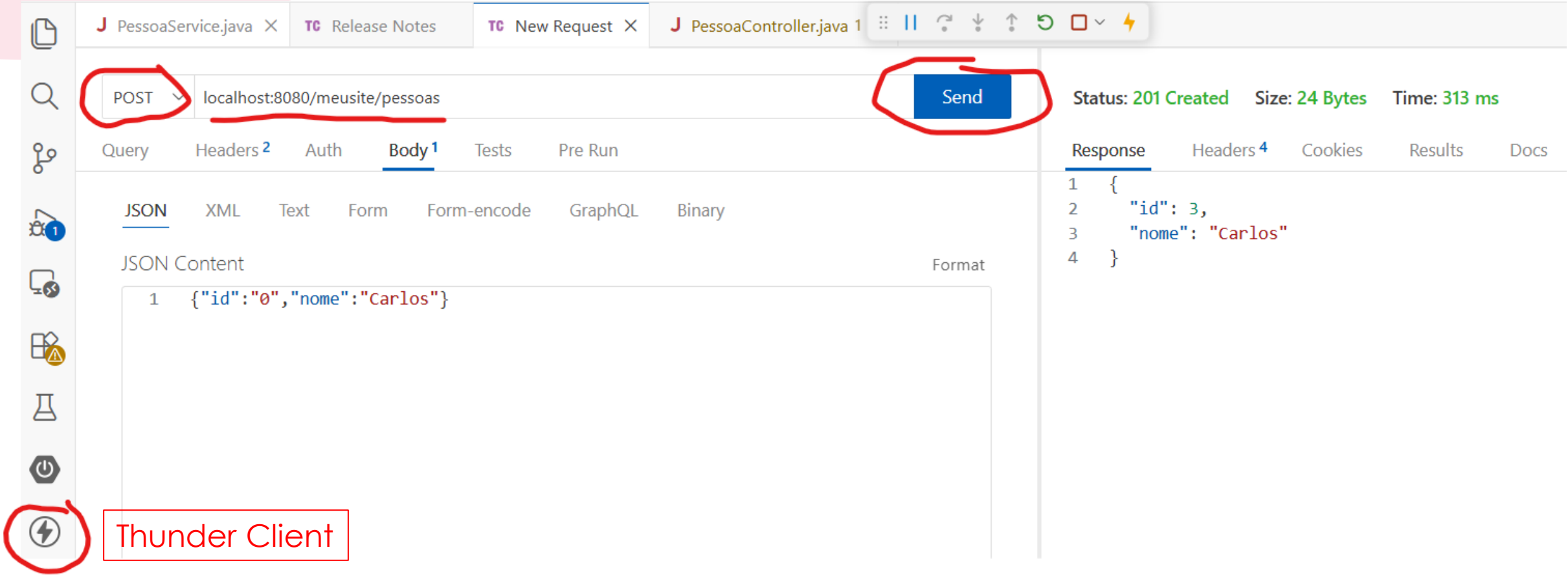
Cria automaticamente objeto Repository.
Mecanismo de injeção de dependência.

Controller

```java
package com.fatecpg.exemplopers.controller;

import org.springframework.web.bind.annotation.RestController;
import com.fatecpg.exemplopers.model.Pessoa;
import com.fatecpg.exemplopers.service.PessoaService;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;

@RestController
@RequestMapping("/meusite")
public class PessoaController {

    private final PessoaService pessoaService;

    public PessoaController(PessoaService pessoaService) {
        this.pessoaService = pessoaService;
    }

    @PostMapping("/pessoas")
    @ResponseStatus(org.springframework.http.HttpStatus.CREATED) // status 201 = Created
    public ResponseEntity<Pessoa> postMethodName(@RequestBody Pessoa pessoa) {
        Pessoa pessoaCriada = pessoaService.salvarPessoa(pessoa);
        return ResponseEntity.ok(pessoaCriada);
    }

}
```

Cria automaticamente objeto service.
Mecanismo de injeção de dependência.

Gerar requisição Post – Cliente de API



Tratamento geral de exceções

```java
package com.fatecpg.exemplopers.exception;

import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.Map;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.context.request.WebRequest;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(Exception.class)
    public ResponseEntity<Object> handleException(Exception ex, WebRequest request) {
        Map<String, Object> body = new HashMap<>();
        body.put("timestamp", LocalDateTime.now());
        body.put("message", ex.getMessage());
        body.put("details", request.getDescription(false));
        return new ResponseEntity<>(body, HttpStatus.INTERNAL_SERVER_ERROR);
    }

    @ExceptionHandler(DataIntegrityViolationException.class)
    public ResponseEntity<Object> handleDataIntegrityViolationException(Exception ex, WebRequest request) {
        Map<String, Object> body = new HashMap<>();
        body.put("timestamp", LocalDateTime.now());
        body.put("message", "Violação de integridade de dados.");
        body.put("details", request.getDescription(false));

        return new ResponseEntity<>(body, HttpStatus.CONFLICT);
    }
}
```

Consultar todas as pessoas

Incluir em Service

```
public List<Pessoa> getAllPessoas() {
    return pessoaRepository.findAll();
}
```

Incluir em Controller

```
@GetMapping("/pessoas")
    public ResponseEntity<List<Pessoa>> allPessoas() {
        List<Pessoa> pessoas = pessoaService.getAllPessoas();
        return ResponseEntity.ok(pessoas);
    }
```

Atualizar dados de pessoas cadastradas

Incluir em Service

```
public Optional<Pessoa> updatePessoa(Long id, Pessoa newPessoaData) {
    return pessoaRepository.findById(id).map(pessoa -> {
        pessoa.setNome(newPessoaData.getNome());
        return pessoaRepository.save(pessoa);
    });
}
```

Incluir em Controller

```
@PutMapping("/pessoas/{id}")
public ResponseEntity<Pessoa> updateUser(@PathVariable Long id, @RequestBody Pessoa pessoa) {
        return pessoaService.updatePessoa(id, pessoa)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }
```

Excluir pessoas

Incluir em Service

```
public boolean deletePessoa(Long id) {
        if (pessoaRepository.existsById(id)) {
            pessoaRepository.deleteById(id);
            return true;
        }
        return false;
    }
```

Incluir em Controller

```
@DeleteMapping("/pessoas/{id}")
    public ResponseEntity<Void> deletePessoa(@PathVariable Long id) {
        boolean deleted = pessoaService.deletePessoa(id);
        if (deleted) {
            return ResponseEntity.noContent().build(); // 204 No Content
        } else {
            return ResponseEntity.notFound().build(); // 404 Not Found
        }
    }
```

Consultar todas as pessoas

GET localhost:8080/meusite/pessoas Send

Query Headers 2 Auth Body 1 Tests Pre Run

Query Parameters

parameter value

Status: 200 OK Size: 75 Bytes Time: 384 ms

Response Headers 4 Cookies Results Docs


```
1  [
2    {
3      "id": 1,
4      "nome": "Carlos"
5    },
6    {
7      "id": 6,
8      "nome": "Maria"
9    },
10   {
11     "id": 15,
12     "nome": "Mario"
13   }
14 ]
```

Atualizar pessoas

PUT localhost:8080/meusite/pessoas/6 Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1  {"id":"0","nome":"Mariana"}
```

Status: 200 OK Size: 25 Bytes Time: 150 ms

Response Headers 4 Cookies Results Docs

```
1  {
2    "id": 6,
3    "nome": "Mariana"
4  }
```

Excluir pessoas

DELETE | localhost:8080/meusite/pessoas/6 | Send

Query | Headers 2 | Auth | Body 1 | Tests | Pre Run

Query Parameters

parameter	value

Status: 204 No Content Size: 0 Bytes Time: 460 ms

Response | Headers 2 | Cookies | Results | Docs

1